CS301: Silberschatz - Operating System Concepts (Ch. 8)

Chapter 8: Deadlocks

8.1 System Model & Foundational Definitions

A computer system consists of a finite number of resources to be distributed among a number of competing processes. Resources are categorized into several types, each consisting of some number of identical instances (CPU cycles, memory, files, printers). A process must request a resource before using it and must release the resource after finishing. A set of processes is in an anomalous deadlock state when every process in the set is waiting for an event that can only be caused by another process in the set.

CS301: Silberschatz - Operating System Concepts (Ch. 8)

8.2 Deadlock Characterization: The Four Coffman Conditions

A deadlock situation can arise if and only if the following four conditions hold simultaneously in a concurrent computing system:

1. Mutual Exclusion: At least one resource must be held in a nonsharable mode.
2. Hold and Wait: A process must be simultaneously holding at least one resource and actively requesting additional resources held by other executing processes.
3. No Preemption: Resources cannot be forcibly preempted; a resource can be released only voluntarily by the process holding it after that process completes its task.
4. Circular Wait: A closed set {P0, P1, ..., Pn} of waiting processes must exist such that P0 is waiting for a resource held by P1, P1 is waiting for P2, and Pn is waiting for P0.

CS301: Silberschatz - Operating System Concepts (Ch. 8)

8.3 Resource-Allocation Graphs & Deadlock vs Starvation

"A deadlock occurs when every process in a set is waiting for an event that only another process in the set can caus

"In contrast, starvation (indefinite postponement) occurs when a runnable process is perpetually denied access bec

It is vital to distinguish deadlock from starvation:

- Deadlock represents a cyclic dependency where progress is mathematically impossible without intervention.
- Starvation represents an unfair scheduling outcome where progress is delayed indefinitely,
  even though no circular wait exists and the system as a whole continues executing tasks.

If a resource allocation graph contains cycles and each resource type has only one instance,
then a deadlock has definitely occurred. With multiple instances per resource type, a cycle
indicates a potential deadlock, but not a guaranteed deadlock condition.

CS301: Silberschatz - Operating System Concepts (Ch. 8)

8.4 Deadlock Avoidance & The Banker’s Algorithm

Deadlock avoidance algorithms require that the operating system be given priori information concerning which resources a process will request throughout its lifetime.
The Banker’s Algorithm ensures the system never enters an unsafe state.
A state is considered safe if the system can allocate resources to each process (up to its maximum) in some specified order without ever triggering a deadlock.
Safety check condition: Allocation + Need <= Max resources available.